

UNIVERSITÉ DE LORRAINE

Rapport de Preuve et Dédution Automatique

Implémentation de la méthode des connexions en Prolog

Étudiants :

Mathéo VIGNÈRES

Guillaume POTEL

Enseignant :

M. Didier GALMICHE

Année universitaire 2025-2026

Table des matières

Introduction	2
1 Le socle commun : Logique Propositionnelle et Premier Ordre	2
1.1 Définition des règles (Alpha et Bêta)	2
1.1.1 Le prédicat principal : <code>etiqueter/7</code>	2
1.1.2 Les règles de type Alpha (α)	2
1.1.3 Les règles de type Bêta (β)	2
1.1.4 Cas de la négation et des feuilles	3
1.2 Structures de données : Représentation des arbres	3
1.2.1 Une structure récursive uniforme	3
1.2.2 Polyvalence de la structure	4
1.3 Prédicats utilitaires	4
1.3.1 Identification des littéraux et opérateurs	4
1.3.2 Substitution de termes	4
1.3.3 Gestion de l’affichage et de la trace	5
2 En Logique Propositionnelle	5
2.1 Construction de l’arbre syntaxique indexé	5
2.2 Génération de l’arbre des chemins	5
2.3 Recherche de connexions et validation	5
3 En Logique du Premier Ordre (LPO)	5
3.1 Adaptation de l’arbre indexé	5
3.2 Évolution de l’arbre des chemins	5
3.3 Recherche de connexions orientée LPO	5
3.4 Unification, graphe de dépendance et multiplicité	5
Conclusion	5

Introduction

1 Le socle commun : Logique Propositionnelle et Premier Ordre

Dans cette première partie, nous présentons les éléments de base qui sont partagés entre la logique propositionnelle et la logique du premier ordre.

1.1 Définition des règles (Alpha et Bêta)

La méthode des connexions s'appuie sur l'application de règles appelée *alpha* et *beta*. Nous avons implémenté ces règles via un prédicat central : `etiqueter/7`.

Ce prédicat a pour rôle de parcourir la formule initiale pour construire un arbre indexé, où chaque nœud contient le type de règle appliquée ($\alpha, \beta, \gamma, \delta$), sa polarité, et un index unique.

1.1.1 Le prédicat principal : `etiqueter/7`

La signature du prédicat est la suivante :

```
1 etiqueter(+Formule, +Polarite, +IndexIn, -IndexOut, +Multiplicite,  
+Suffixe, -Arbre).
```

- **Polarité** : 1 pour une formule affirmée, 0 pour une formule niée.
- **IndexIn / IndexOut** : Un mécanisme de compteur permettant d'attribuer un identifiant unique (ex : $a0, a1, \dots$) à chaque nœud de l'arbre.
- **Arbre** : La structure de sortie représentant l'arbre syntaxique indexé sous forme de nœud ou de feuille.

1.1.2 Les règles de type Alpha (α)

Les règles α correspondent aux décompositions de nature conjonctive (linéaire). Dans notre implémentation, une règle α produit deux sous-arbres qui devront tous deux être validés. Par exemple, pour la conjonction de polarité 1 ($A \wedge B, 1$) ou l'implication de polarité 0 ($A \rightarrow B, 0$) :

```
1 % Exemple : Implication niee (A -> B, 0)  
2 etiqueter(A impl B, 0, IndexIn, IndexOut, M, Suffix,  
3     noeud(etiq_formule(alpha, alpha1, alpha2, A impl B, 0,  
4         NomComplet),  
5         [ArbreA, ArbreB])) :-  
6     construire_nom(IndexIn, Suffix, NomComplet),  
7     Index1 is IndexIn + 1,  
8     etiqueter(A, 1, Index1, IndexMid, M, Suffix, ArbreA),  
     etiqueter(B, 0, IndexMid, IndexOut, M, Suffix, ArbreB).
```

1.1.3 Les règles de type Bêta (β)

Les règles β représentent des décompositions disjonctives (branchement). Elles créent une bifurcation dans l'arbre des chemins. Nous retrouvons ici la disjonction de polarité 1 ($A \vee B, 1$) ou l'implication de polarité 0 ($A \rightarrow B, 1$) :

```

1 % Exemple : Disjonction affirmée (A ou B, 1)
2 etiqueter(A ou B, 1, IndexIn, IndexOut, M, Suffix,
3     noeud(etiq_formule(beta, beta1, beta2, A ou B, 1,
4         NomComplet),
5         [ArbreA, ArbreB])) :-
6     construire_nom(IndexIn, Suffix, NomComplet),
7     Index1 is IndexIn + 1,
8     etiqueter(A, 1, Index1, IndexMid, M, Suffix, ArbreA),
9     etiqueter(B, 1, IndexMid, IndexOut, M, Suffix, ArbreB).

```

1.1.4 Cas de la négation et des feuilles

La négation est traitée de manière transparente en inversant la polarité du reste de la formule. Ce processus se poursuit jusqu'à atteindre un **littéral** (atome). À ce stade, le prédicat crée une feuille, marquant la fin de la branche syntaxique.

```

1 % Traitement des atomes (Feuilles)
2 etiqueter(A, Polarite, Index, IndexOut, _, Suffix,
3     feuille(etiq_formule(atome, none, none, A, Polarite,
4         NomComplet)))) :-
5     est_litteral(A),
6     construire_nom(Index, Suffix, NomComplet),
7     IndexOut is Index + 1.

```

1.2 Structures de données : Représentation des arbres

Pour représenter un arbre dans notre projet, nous avons choisi une structure récursive polyvalente capable de représenter aussi bien l'arbre syntaxique indexé que l'arbre des chemins.

1.2.1 Une structure récursive uniforme

Un arbre est défini de manière inductive par deux constructeurs principaux :

- `noeud(Etiquette, ListeFils)` : Représente un point de divergence ou de linéarité dans la formule. Contrairement à un arbre binaire classique, nous utilisons une liste de fils pour permettre une plus grande flexibilité (notamment pour la gestion de la multiplicité en LPO).
- `feuille(Etiquette)` : Représente un point terminal, (un littéral).

L'étiquette contient les métadonnées nécessaires au traitement : pour l'arbre syntaxique, il s'agit du prédicat `etiq_formule` vu précédemment. Pour l'arbre des chemins, elle contient l'ensemble des littéraux collectés.

```

1 % Verification du type de noeud
2 est_feuille(feuille(_)).
3 est_noeud(noeud(_, _)).
4
5 % Accesseurs pour l'extraction de donnees
6 noeud_etiquette(noeud(E, _), E).
7 noeud_fils(noeud(_, Fils), Fils).
8 feuille_etiquette(feuille(E), E).

```

1.2.2 Polyvalence de la structure

Cette implémentation est utilisée à deux étapes clés du programme :

1. **L'Arbre Indexé** : Chaque nœud correspond à une sous-formule et chaque feuille à un atome avec sa polarité. C'est la base de la décomposition logique.
2. **L'Arbre des Chemins** : Il s'agit d'une transformation de l'arbre précédent où chaque nœud représente l'état des chemins (la "matrice") à un point donné de l'exploration.

Cette uniformité de structure facilite l'écriture des algorithmes de parcours de profondeur, car les mêmes prédicats de navigation peuvent être réutilisés sur les deux types d'arbres.

1.3 Prédicats utilitaires

Pour soutenir les étapes de décomposition et de recherche de preuves, nous avons développé un ensemble d'outils. Ces prédicats gèrent la syntaxe logique, la manipulation des variables et les mécanismes d'affichage.

1.3.1 Identification des littéraux et opérateurs

Un point crucial de la méthode des connexions est de savoir quand arrêter la décomposition d'une formule. Le prédicat `est_litteral/1` permet d'identifier si un terme est un atome (propositionnel ou prédicat du premier ordre) en vérifiant qu'il n'est pas un connecteur logique réservé.

```
1 % Un terme est un littéral s'il n'est pas un connecteur logique
2 est_litteral(A) :-
3     atom(A), !.
4 est_litteral(A) :-
5     compound(A),
6     functor(A, Foncteur, _),
7     \+ connecteur(Foncteur).
```

Nous avons également défini les priorités d'opérateurs pour permettre une écriture naturelle des formules (ex : `p et q impl r`).

1.3.2 Substitution de termes

Particulièrement utilisé en logique du premier ordre pour l'application des règles γ (universelles) et δ (existentielles), le prédicat `substituer/4` effectue un remplacement récursif d'un terme par un autre.

```
1 % Remplacement récursif de Ancien par Nouveau dans Terme
2 substituer(Ancien, Ancien, Nouveau, Nouveau) :- !.
3 substituer(Terme, Ancien, Nouveau, Resultat) :-
4     compound(Terme),
5     Terme =.. [F | Args],
6     maplist([Arg, ArgSubst]>>(substituer(Arg, Ancien, Nouveau,
7     ArgSubst))), Args, ArgsSubst),
8     Resultat =.. [F | ArgsSubst].
9 substituer(Terme, _, _, Terme).
```

Ce prédicat utilise l'opérateur "univ" ($=.$) pour décomposer n'importe quel prédicat en une liste de ses arguments, permettant une substitution générique quelle que soit l'arité du terme.

1.3.3 Gestion de l'affichage et de la trace

Nous avons créé tout un module d'affichage pour suivre les traces de la méthode des connexions. Pour ne pas encombrer la logique métier, ces outils sont regroupés et traitent les points suivants :

- **Écriture symbolique** : Les prédicats `ecrire_formule/1` et `ecrire_type/1` traduisent les termes internes (ex : `impl`, `alpha`) en symboles mathématiques ($\rightarrow$, α , $\forall$, $\exists$).
- **Nettoyage des variables** : `nettoyer_formule/2` transforme les structures internes complexes (utilisées pour l'unification) en noms lisibles pour l'utilisateur final.
- **Système d'Echo** : Nous avons réutilisé le système d'affichage donné pour le projet d'algorithme d'unification de Martelli-Montanari au 1er semestre, dans le cadre de l'UE *Logique et Modèles de Calculs*.

2 En Logique Propositionnelle

Cette section détaille l'implémentation de la méthode des connexions restreinte à la logique propositionnelle.

2.1 Construction de l'arbre syntaxique indexé

2.2 Génération de l'arbre des chemins

2.3 Recherche de connexions et validation

3 En Logique du Premier Ordre (LPO)

La logique du premier ordre introduit la gestion des quantificateurs, ce qui complexifie significativement la méthode des connexions.

3.1 Adaptation de l'arbre indexé

3.2 Évolution de l'arbre des chemins

3.3 Recherche de connexions orientée LPO

3.4 Unification, graphe de dépendance et multiplicité

Conclusion